Evaluating and Refining Arch-Stage

Supporting Architectural Hypotheses in Software Engineering Teams

José Gonçalves Lima Neto

December 2026

Master's Thesis Defense

Graduate Program in Computer Science
IME – University of São Paulo

CC BY

Overview

1. Introduction
2. Background
3. Related Work
4. Research Design
5. Results
6. Conclusion and Future Work
7. References

Overview

1. Introduction
2. Background
3. Related Work
4. Research Design
5. Results
6. Conclusion and Future Work
7. References

Motivation

- Organizations strive for fast flow through autonomous teams
- Architecture decisions often remain centralized in a few experts
- Architectural uncertainty is natural but usually implicit

- Existing derivation methods rely on architects' tacit knowledge (Souza et al., 2019)
- Teams without senior architects lack guidance and tooling

The Problem

Gap

No consolidated approach exists for managing architectural uncertainty within the daily engineering workflow of development teams.

- **ArchHypo (Silva et al., TSE 2025) uses hypotheses engineering to make uncertainty explicit and manageable**
- **But: manual application has a steep learning curve (Finding 9: unanimously hard to learn)**
- **Process-change resistance (Finding 8) hinders adoption**

Objective

Thesis Objective

Empirically evaluate and refine **Arch-Stage**, a support tool for the ArchHypo technique, by applying it with diverse software engineering teams in a real-world organization with a scaled software product.

Overview

1. Introduction
2. Background
3. Related Work
4. Research Design
5. Results
6. Conclusion and Future Work
7. References

ArchHypo Technique

1. Identify architectural hypotheses (uncertainties)
2. Assess each: uncertainty level + impact level
3. Plan technical actions (spikes, tracer bullets, triggers)
4. Execute plans and reassess iteratively

- Prior evaluations: Catch Solve startup (IEEE SW), PropSEP at Jetsoft (TSE)
- Both manual — no tool support

Arch-Stage Tool

- **Open-source Backstage plugin for ArchHypo**
- **Hypothesis CRUD, Likert-scale assessment, technical plans**
- **Visualization: uncertainty vs. impact chart**
- **Integrates into the Internal Developer Portal workflow**

Lineage

Initial design & implementation: undergrad capstone (Corrêa, 2025).

This thesis: empirical evaluation & refinement with real teams.

Overview

1. Introduction
2. Background
3. **Related Work**
4. Research Design
5. Results
6. Conclusion and Future Work
7. References

Related Work

- ADR tools (adr-tools, Log4brains) — capture decisions, not uncertainties
- ATAM workshops — heavyweight, not continuous
- IDPs (Backstage, Port, Cortex) — no native hypothesis support

- Gap: no existing tool combines hypothesis engineering with developer portal integration

Overview

1. Introduction
2. Background
3. Related Work
4. **Research Design**
5. Results
6. Conclusion and Future Work
7. References

Empirical Approach

- **Context: CompanyX, scaled product, multiple teams**
- **Method: embedded case study, mixed methods**
- **Data sources:**
  - Arch-Stage artifacts (hypotheses, assessments, plans)
  - Questionnaires (usability, perceived value)
  - Semi-structured interviews
- **Analysis: thematic coding + descriptive statistics**

Overview

1 Introduction
2 Background
3 Related Work
4 Research Design
5 Results
6 Conclusion and Future Work
7 References

Results

(to be presented after the empirical study)

Overview

1. Introduction
2. Background
3. Related Work
4. Research Design
5. Results
6. **Conclusion and Future Work**
7. References

Conclusion and Future Work

- Summary of contributions
- Limitations
- Future directions

Overview

1. Introduction
2. Background
3. Related Work
4. Research Design
5. Results
6. Conclusion and Future Work
7. References

References i

- Bass, Len et al. (2021). *Software Architecture in Practice*. 4th ed. Addison-Wesley.
- Corrêa, Pedro Henrique Mariano (2025). *ArchHypo Tooling: Enabling Practical Hypothesis Engineering for Software Architecture*. Capstone Project Report (Bachelor), Institute of Mathematics and Statistics, University of São Paulo. Supervised by José G. Lima Neto; co-supervised by Paulo Meirelles.
- Foote, Brian and Joseph Yoder (1999). "Big Ball of Mud". *Pattern Languages of Program Design* 4, pp. 654–692.
- Richards, Mark and Neal Ford (2020). *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media. ISBN: 9781492043454.
- Silva, Eduardo, Jorge Melegati, Moises Silveira, et al. (2025). "ArchHypo: Managing Software Architecture Uncertainty Using Hypotheses Engineering". *IEEE Transactions on Software Engineering* 51.2, pp. 481–499. DOI: 10.1109/TSE.2024.3520477.

References ii

- Silva, Eduardo, Jorge Melegati, Xiaofeng Wang, et al. (2024). "Using Hypotheses to Manage Technical Uncertainty and Architecture Evolution in a Software Start-up". *IEEE Software* 41.4, pp. 78–87. DOI: 10.1109/MS.2024.3383628.
- Skelton, Matthew and Manuel Pais (2019). *Team Topologies: Organizing Business and Technology Teams for Fast Flow*. IT Revolution Press.
- Souza, Eric et al. (2019). "Deriving Architectural Models from Requirements Specifications: A Systematic Mapping Study". *Information and Software Technology* 109, pp. 26–39. DOI: 10.1016/j.infsof.2019.01.004.

Evaluating Arch-Stage

1. Introduction
2. Background
3. Related Work
4. Research Design
5. Results
6. Conclusion and Future Work
7. References

https://github.com/ArchHypo/hypo-stage-masters-thesis

Backup: ArchHypo Nine Findings (TSE)

1. Some hypotheses monitored without full resolution until end
2. Avoided upfront architectural design; decisions through iterations
3. Improved process: safety, transparency, predictability
4. Different QAs handled with different creative strategies
5. Guideline implementation was the most-used strategy
6. Team recognized better architectural decision-making
7. Evidence of positive impact on product quality
8. **Process-change obstacle**: mixing arch & functional tasks
9. **Unanimously hard to learn**: mapping, articulating, planning

Backup: Souza et al. Key Findings

- 2575 papers → 39 primary studies
- All methods rely on architects' tacit knowledge
- Limited support for inexperienced practitioners
- Weak or absent tool support
- Insufficient empirical evaluation